

HOISTING & EXECUTION CONTEXT IN JAVASCRIPT



Lakshmiprasanna Vakada

Hoisting

Hoisting is JavaScript's default behavior of moving function and variable declarations to the top of their scope before execution.

✓ Hoisting with var (Gets Undefined)

```
console.log(name); // ? undefined  
var name = "John";  
console.log(name); // ✓ John
```

The declaration (var name) is hoisted, but the assignment ("John") is NOT hoisted.

✓ Hoisting with let & const (Reference Error)

```
console.log(age); // ✗ ReferenceError  
let age = 25;
```

let and const are hoisted, but they are in a Temporal Dead Zone (TDZ) until initialized.

✓ Hoisting with Functions

```
greet(); // ✓ "Hello!"  
  
function greet() {  
  console.log("Hello!");  
}
```

Function declarations are fully hoisted, meaning you can call them before defining.

Execution Context in JavaScript

An execution context is the environment where JavaScript code runs.

Every JavaScript program has at least one execution context (Global Execution Context).

Types of Execution Context

- ✓ Global Execution Context (GEC) – Created when the script starts.
- ✓ Function Execution Context (FEC) – Created every time a function is called.
- ✓ Eval Execution Context – Created when `eval()` is used (rarely).

Call Stack

JavaScript follows Last In, First Out (LIFO) to execute functions.

Each time a function is called, it is pushed onto the stack, and once completed, it is popped off.

✓ Example: Call Stack in Action

```
function first() {  
    console.log("First");  
}  
function second() {  
    first(); // Calls first()  
    console.log("Second");  
}  
second();  
console.log("End");
```

◆ Execution Order:

- 1 second() is called → Pushed to stack
- 2 first() is called inside second() → Pushed to stack
- 3 first() completes → Popped from stack
- 4 second() completes → Popped from stack
- 5 "End" is printed

Lexical Environment & Scope Chain

A Lexical Environment is where a function is physically written in the code and determines which variables it can access.

✓ Example: Lexical Scope in Action

```
function outer() {  
  let outerVar = "I am outer!";  
  
  function inner() {  
    console.log(outerVar); // ✓ Can access outerVar  
  }  
  
  inner();  
}  
  
outer();
```

◆ inner() has access to outerVar because of lexical scope.

Scope Chain in Action

```
let globalVar = "I am global!";

function outer() {
  let outerVar = "I am outer!";

  function inner() {
    let innerVar = "I am inner!";
    console.log(globalVar); //  Accesses global
scope
    console.log(outerVar); //  Accesses outer scope
    console.log(innerVar); //  Accesses own scope
  }

  inner();
}

outer();
```

If JavaScript doesn't find a variable in the current scope, it looks up the chain (Inner → Outer → Global).